

```
In [ ]: import marimo as mo
import csv
from ortools.sat.python import cp_model
import time
from collections import Counter
```

```
Traceback (most recent call last):
  File "", line 3, in <module>
    from ortools.sat.python import cp_model
ModuleNotFoundError: No module named 'ortools'
```

Gerador de horário escolar

1. O problema e a abordagem

Queremos montar o horário semanal de duas turmas, com 5 dias e 5 tempos por dia, respeitando a carga de cada disciplina, a disponibilidade dos professores e o número de salas. Quando os recursos mudam um pouco, o horário deve ser ajustado sem ser refeito de raiz.

Tratámos isto como um problema de restrições e usámos o CP-SAT, do OR-Tools, que é o que a disciplina sugere. A ideia é descrever o que um horário válido tem de cumprir e deixar o solver encontrar um. Para cada turma, disciplina, dia e tempo existe uma variável de sim ou não ("esta turma tem esta disciplina neste tempo?"), e cada requisito do enunciado é uma regra sobre essas variáveis.

```
In [ ]: # 0 enunciado fixa a grelha em 5 dias com 5 tempos, por isso estes dois
        # ficam
        # escritos aqui. Tudo o resto vem dos CSV.

dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
tempos = list(range(1, 6))
```

2. Dados

Os dados vêm de quatro ficheiros CSV (turmas, disciplinas, salas e exceções de disponibilidade) e são lidos por uma função que recebe a pasta como argumento. Assim, a mesma leitura serve para `dados/`, `dados_v2/` e para qualquer outra pasta com o mesmo formato, e não há dados escritos no código (R8).

Só os dias e os tempos estão no código, porque o enunciado fixa a grelha em 5 dias com 5 tempos.

```
In [ ]: # R8. Lê os quatro CSV de uma pasta. Está numa função para usarmos a mesma
        # leitura com dados/, dados_v2/ e dados_teste/.
```

```

def ler_dados(pasta):
    with open(f"{pasta}/turmas.csv", encoding="utf-8") as f:
        turmas = [linha["turma"] for linha in csv.DictReader(f)]
    with open(f"{pasta}/disciplinas.csv", encoding="utf-8") as f:
        disciplinas = [
            {
                "disciplina": linha["disciplina"],
                "professor": linha["professor"],
                "carga_semanal": int(linha["carga_semanal"]),
                "duplo_perodo": linha["duplo_perodo"],
                "sala_especial": linha["sala_especial"],
            }
            for linha in csv.DictReader(f)
        ]
    with open(f"{pasta}/salas.csv", encoding="utf-8") as f:
        salas = [
            {
                "sala": linha["sala"],
                "tipo": linha["tipo"],
                "quantidade": int(linha["quantidade"]),
            }
            for linha in csv.DictReader(f)
        ]
    with open(f"{pasta}/disponibilidade_excecoes.csv", encoding="utf-8") as f:
        disponibilidade_excecoes = [
            {
                "professor": linha["professor"],
                "dia": linha["dia"],
                "periodo": int(linha["periodo"]),
            }
            for linha in csv.DictReader(f)
        ]

    return turmas, disciplinas, salas, disponibilidade_excecoes

```

```
In [ ]: turmas, disciplinas, salas, disponibilidade_excecoes = ler_dados("dados")
```

3. O modelo

A função `construir_modelo` recebe os dados e devolve o modelo com as variáveis e as regras. Cada variável é indexada por turma, disciplina, dia e tempo (300, com os dados iniciais). O professor não faz parte do índice, porque cada disciplina tem um só professor e sabemos qual é a partir da disciplina.

Regra	Como está escrita
R1	em cada tempo, uma turma tem no máximo uma aula
R2	na semana, cada disciplina tem exatamente a carga definida

Regra	Como está escrita
R3	por dia, no máximo uma aula da disciplina (dois tempos, se for duplo período)
R4	no duplo período, cada tempo com aula tem um vizinho com aula
R5	em cada tempo, um professor tem no máximo uma aula
R6	nos tempos de exceção, as aulas do professor somam zero
R7	em cada tempo, as aulas que usam uma sala não passam da quantidade dessa sala

O objetivo O1, minimizar os buracos, também é construído aqui. Para cada professor, dia e tempo, um buraco é um tempo vazio que tem aulas antes e depois.

```
In [ ]: # É aqui que está o problema todo: uma variável sim/não por cada
# (turma, disciplina, dia, tempo) e uma regra por requisito, de R1 a R7.
# Recebe os dados como parâmetros porque precisamos de o construir mais do
# que uma vez.

def construir_modelo(turmas, disciplinas, salas, excecoes):
    nomes = [d["disciplina"] for d in disciplinas]
    professores = sorted(set(d["professor"] for d in disciplinas))
    modelo = cp_model.CpModel()

    # variáveis: x[turma, disciplina, dia, tempo] = 1 se há aula
    x = {}
    for turma in turmas:
        for disc in nomes:
            for dia in dias:
                for tempo in tempos:
                    x[turma, disc, dia, tempo] = modelo.new_bool_var(
                        f"x_{turma}_{disc}_{dia}_{tempo}")

    # R1: uma turma não tem duas aulas ao mesmo tempo
    for turma in turmas:
        for dia in dias:
            for tempo in tempos:
                modelo.add(sum(x[turma, disc, dia, tempo] for disc in
                                nomes) <= 1)

    # R2: carga semanal exata
    for turma in turmas:
        for d in disciplinas:
            modelo.add(
                sum(x[turma, d["disciplina"], dia, tempo]
                    for dia in dias for tempo in tempos) ==
                d["carga_semanal"])

    # R3: no máximo uma aula da disciplina por dia (um bloco de 2, se
    # duplo)
    for turma in turmas:
        for d in disciplinas:
            limite = 2 if d["duplo_periodo"] == "sim" else 1
            for dia in dias:
```

```

        modelo.add(
            sum(x[turma, d["disciplina"], dia, tempo] for tempo in
tempos) <= limite)

    # R4: duplo período -> cada tempo com aula tem de ter um vizinho com
aula
    for turma in turmas:
        for d in disciplinas:
            for dia in dias:
                for tempo in tempos:
                    if d["duplo_periodo"] == "sim":
                        modelo.add(
                            x[turma, d["disciplina"], dia, tempo]
                            <= sum(x[turma, d["disciplina"], dia, v]
                                for v in (tempo - 1, tempo + 1) if v in
tempos))

    # R5: um professor não dá duas aulas ao mesmo tempo
    for prof in professores:
        for dia in dias:
            for tempo in tempos:
                modelo.add(
                    sum(x[turma, d["disciplina"], dia, tempo]
                        for turma in turmas for d in disciplinas
                        if d["professor"] == prof) <= 1)

    # R6: indisponibilidades dos professores
    for e in excecoes:
        modelo.add(
            sum(x[turma, d["disciplina"], e["dia"], e["periodo"]]
                for turma in turmas for d in disciplinas
                if d["professor"] == e["professor"]) == 0)

    # R7: capacidade das salas
    for sala in salas:
        if sala["tipo"] == "normal":
            usam = [d for d in disciplinas if d["sala_especial"] == ""]
        else:
            usam = [d for d in disciplinas if d["sala_especial"] ==
sala["sala"]]
        for dia in dias:
            for tempo in tempos:
                modelo.add(
                    sum(x[turma, d["disciplina"], dia, tempo]
                        for turma in turmas for d in usam) <=
sala["quantidade"])

    # 01: buracos. Para cada professor/dia/tempo h:
    #   antes = há aula em algum tempo <= h
    #   depois = há aula em algum tempo >= h
    #   buraco >= antes + depois - 1 - ocupado    (só é 1 se h está vazio e
entre aulas)
    buracos = []
    for prof in professores:
        for dia in dias:
            ocupado = {

```

```

        h: sum(x[turma, d["disciplina"], dia, h]
              for turma in turmas for d in disciplinas
              if d["professor"] == prof)
    for h in tempos
}
for h in tempos:
    antes = modelo.new_bool_var(f"antes_{prof}_{dia}_{h}")
    depois = modelo.new_bool_var(f"depois_{prof}_{dia}_{h}")
    buraco = modelo.new_bool_var(f"buraco_{prof}_{dia}_{h}")
    for k in tempos:
        if k <= h:
            modelo.add(antes >= ocupado[k])
        if k >= h:
            modelo.add(depois >= ocupado[k])
    modelo.add(buraco >= antes + depois - 1 - ocupado[h])
    buracos.append(buraco)

return modelo, x, sum(buracos)

```

4. Resolver e verificar

O `resolver` pede ao solver uma solução e devolve o horário como um conjunto de aulas, cada uma no formato (turma, disciplina, dia, tempo). As funções `gerar_do_zero` e `gerar_incremental`, que usam o `resolver`, são explicadas na secção 5.

Para não depender só do solver, o `verificar` confere um horário olhando apenas para as aulas e para os dados, e devolve a lista de regras violadas (vazia, se o horário for válido). Testámos o verificador com sete horários inválidos de propósito, um por regra de R1 a R7, e ele apanhou todos.

```

In [ ]: # Pede ao solver para resolver o modelo e devolve o horário como um
        # conjunto
        # de aulas (turma, disciplina, dia, tempo). Se não houver solução, devolve
        # None.
def resolver(modelo, x, limite):
    solver = cp_model.CpSolver()
    solver.parameters.max_time_in_seconds = limite
    estado = solver.solve(modelo)
    nome = solver.status_name(estado)
    if nome in ("OPTIMAL", "FEASIBLE"):
        return nome, {k for k in x if solver.value(x[k]) == 1}
    return nome, None

# Conta os buracos de um horário olhando só para as aulas, sem usar o
# solver.
# Para cada professor e dia: (último tempo - primeiro tempo + 1) - tempos
# com aula.
def buracos_totais(horario, disciplinas):
    prof_de = {d["disciplina"]: d["professor"] for d in disciplinas}

```

```

ocupados = {}
for (turma, disc, dia, tempo) in horario:
    ocupados.setdefault((prof_de[disc], dia), set()).add(tempo)
return sum((max(ts) - min(ts) + 1) - len(ts) for ts in
ocupados.values())

# Conta as aulas do horário novo que não estavam no antigo.
def mudancas(antigo, novo):
    return len(novo - antigo)

# Gera um horário do zero: constrói o modelo e minimiza os buracos (01).
def gerar_do_zero(turmas, disciplinas, salas, excecoes, limite=30):
    t0 = time.perf_counter()
    modelo, x, buracos = construir_modelo(turmas, disciplinas, salas,
excecoes)
    modelo.minimize(buracos)
    estado, horario = resolver(modelo, x, limite)
    return estado, horario, time.perf_counter() - t0

# Mesmo modelo, mas começamos pelo horário antigo e pedimos ao solver que
# mexa no menos possível.
def gerar_incremental(anterior, turmas, disciplinas, salas, excecoes,
limite=30):
    t0 = time.perf_counter()
    modelo, x, _ = construir_modelo(turmas, disciplinas, salas, excecoes)
    for k in x:
        modelo.add_hint(x[k], 1 if k in anterior else 0)
    modelo.maximize(sum(x[k] for k in anterior if k in x))
    estado, horario = resolver(modelo, x, limite)
    return estado, horario, time.perf_counter() - t0

```

```

In [ ]: # Confere um horário só com os dados, sem passar pelo solver. Se houver um
# erro nas regras do modelo, aqui não se repete.
def verificar(horario, turmas, disciplinas, salas, excecoes):
    erros = []
    info = {d["disciplina"]: d for d in disciplinas}
    prof_de = {d["disciplina"]: d["professor"] for d in disciplinas}

    for (turma, disc, dia, tempo) in horario:
        if turma not in turmas or disc not in info or dia not in dias or
tempo not in tempos:
            erros.append(f"Aula inválida: {(turma, disc, dia, tempo)}")
    if erros:
        return erros

    # R1: uma turma não tem duas aulas ao mesmo tempo
    for (turma, dia, tempo), n in Counter((t, di, h) for (t, s, di, h) in
horario).items():
        if n > 1:
            erros.append(f"R1: {turma} tem {n} aulas em {dia}, tempo
{tempo}")

```

```

# R2: carga semanal exata
por_disc = Counter((t, s) for (t, s, di, h) in horario)
for turma in turmas:
    for d in disciplinas:
        n = por_disc[(turma, d["disciplina"])]
        if n != d["carga_semanal"]:
            erros.append(f"R2: {turma} tem {n} aulas de
{d['disciplina']} "
                        f"(carga {d['carga_semanal']})")

# R3 e R4: uma aula por dia; no duplo período, um bloco de 2 tempos
seguidos
tempos_do_dia = {}
for (t, s, di, h) in horario:
    tempos_do_dia.setdefault((t, s, di), []).append(h)
for (turma, disc, dia), ts in tempos_do_dia.items():
    ts = sorted(ts)
    if info[disc]["duplo_periodo"] == "sim":
        if len(ts) > 2:
            erros.append(f"R3: {turma} tem {len(ts)} tempos de {disc}
em {dia}")
        elif len(ts) == 1 or (len(ts) == 2 and ts[1] - ts[0] != 1):
            erros.append(f"R4: {turma}, {disc} em {dia} não é um bloco
de 2 seguidos: {ts}")
        elif len(ts) > 1:
            erros.append(f"R3: {turma} tem {len(ts)} aulas de {disc} em
{dia}")

# R5: um professor não dá duas aulas ao mesmo tempo
for (prof, dia, tempo), n in Counter(
    (prof_de[s], di, h) for (t, s, di, h) in horario).items():
    if n > 1:
        erros.append(f"R5: {prof} tem {n} aulas em {dia}, tempo
{tempo}")

# R6: nenhuma aula num tempo em que o professor está indisponível
for e in excecoes:
    for (t, s, di, h) in horario:
        if prof_de[s] == e["professor"] and di == e["dia"] and h ==
e["periodo"]:
            erros.append(f"R6: {e['professor']} tem aula em {di}, tempo
{h}, "
                        f"mas está indisponível")

# R7: capacidade das salas
cap_normal = sum(s["quantidade"] for s in salas if s["tipo"] ==
"normal")
cap_especial = {s["sala"]: s["quantidade"] for s in salas if s["tipo"]
== "especial"}
uso = Counter()
for (t, s, di, h) in horario:
    sala = info[s]["sala_especial"] or "normal"
    uso[(sala, di, h)] += 1
for (sala, dia, tempo), n in uso.items():
    cap = cap_normal if sala == "normal" else cap_especial.get(sala, 0)
    if n > cap:

```

```

        erros.append(f"R7: {n} aulas em sala '{sala}' em {dia}, tempo
{tempo} (cap. {cap})")

    return erros

```

```

In [ ]: # Cada horário abaixo é inválido de propósito. O verificador tem de
# apanhar a regra certa; se não apanhar, o assert falha.
_invalidos = {
    "R1": {("7ºA", "Matemática", "Seg", 1), ("7ºA", "Português", "Seg",
1)},
    "R2": set(),
    "R3": {("7ºA", "Matemática", "Seg", 1), ("7ºA", "Matemática", "Seg",
2)},
    "R4": {("7ºA", "Educação Física", "Seg", 4), ("7ºA", "Educação Física",
"Seg", 5),
           ("7ºA", "Educação Física", "Ter", 4)},
    "R5": {("7ºA", "História", "Seg", 1), ("7ºB", "Inglês", "Seg", 1)},
    "R6": {("7ºA", "Educação Física", "Seg", 1)},
    "R7": {("7ºA", "Educação Física", "Seg", 4), ("7ºB", "Educação Física",
"Seg", 4)},
}
for _regra, _h in _invalidos.items():
    _erros = verificar(_h, turmas, disciplinas, salas,
disponibilidade_execoes)
    assert any(e.startswith(_regra) for e in _erros), f"{_regra} não foi
detetada"
"todos os testes passaram"

```

5. R9: construção incremental

O H0 é o horário inicial, gerado com os dados de `dados/`. O H1 é o horário para os dados de `dados_v2/` (a Prof. Ana deixa de poder dar aulas à sexta nos dois últimos tempos) e foi gerado de duas maneiras.

Do zero, construímos o modelo com os dados novos e minimizamos os buracos, sem olhar para o H0. No incremental, construímos o mesmo modelo, damos o H0 ao solver como ponto de partida e pedimos que mantenha o máximo de aulas onde estavam.

Escolhemos esta forma porque só depende dos dados novos. Serve para qualquer alteração, seja um professor, uma sala ou uma turma nova, e não precisamos de descobrir à mão que aulas foram afetadas.

A tabela compara as duas maneiras. O que sustenta a conclusão é a coluna das aulas que mudam. Os tempos vêm de uma única execução e variam de uma vez para a outra, por isso só os usamos como indicação. Depois da tabela, confirmamos que H0 e H1 cumprem as regras e comparamos as grelhas da 7ºA.

```

In [ ]: # Fluxo do R9: gera H0 com os dados iniciais e depois H1 com dados_v2,
# de duas maneiras (do zero e incremental), para comparar.

```

```

turmas_v2, disciplinas_v2, salas_v2, excecoes_v2 = ler_dados("dados_v2")

estado_H0, H0, tempo_H0 = gerar_do_zero(
    turmas, disciplinas, salas, disponibilidade_excecoes)
estado_Z, H1_zero, tempo_Z = gerar_do_zero(
    turmas_v2, disciplinas_v2, salas_v2, excecoes_v2)
estado_I, H1, tempo_I = gerar_incremental(
    H0, turmas_v2, disciplinas_v2, salas_v2, excecoes_v2)

mo.md(f"""
| | estado | tempo (s) | aulas que mudam vs H0 | buracos (01) |
|---|---|---|---|---|
| H0 (`dados/`) | {estado_H0} | {tempo_H0:.3f} | - | {buracos_totais(H0,
disciplinas)} |
| H1 do zero (`dados_v2/`) | {estado_Z} | {tempo_Z:.3f} | {mudancas(H0,
H1_zero)} | {buracos_totais(H1_zero, disciplinas_v2)} |
| H1 incremental (`dados_v2/`) | {estado_I} | {tempo_I:.3f} | {mudancas(H0,
H1)} | {buracos_totais(H1, disciplinas_v2)} |
""")

```

```

In [ ]: (verificar(H0, turmas, disciplinas, salas, disponibilidade_excecoes),
verificar(H1, turmas_v2, disciplinas_v2, salas_v2, excecoes_v2))

```

```

In [ ]: # Mostra o horário de uma turma numa grelha: dias nas colunas, tempos nas
linhas.
# As aulas que vierem em `destacar` aparecem a negrito.
def mostrar_horario(horario, turma, destacar=()):
    grelha = {(dia, tempo): (f"***{disc}***" if (t, disc, dia, tempo) in
destacar else disc)
                for (t, disc, dia, tempo) in horario if t == turma}
    linhas = ["| Tempo | " + " | ".join(dias) + " |",
              "|---|" + "---|" * len(dias)]
    for tempo in tempos:
        celulas = [grelha.get((dia, tempo), "") for dia in dias]
        linhas.append(f"| {tempo} | " + " | ".join(celulas) + " |")
    return mo.md("\n".join(linhas))

```

```

In [ ]: # Antes (H0) e depois (H1 incremental), lado a lado. A negrito, a aula
# que saiu de um lado e a que entrou do outro.
mo.vstack([
    mo.hstack([
        mo.vstack([mo.md(f"***{t}: antes (H0)***"), mostrar_horario(H0, t, H0
- H1)]),
        mo.vstack([mo.md(f"***{t}: depois (H1)***"), mostrar_horario(H1, t,
H1 - H0)]),
    ])
    for t in turmas
])

```

6. Outros cenários e testes

Para não depender só do `dados_v2`, experimentámos outra alteração: o professor da primeira aula do H0 fica indisponível nesse tempo. A tabela compara de novo as duas maneiras.

Por fim, corremos o código com uma pasta de dados diferente, `dados_teste/`, com mais uma turma, mais uma disciplina e mais uma exceção. O horário gerado cumpre as regras, o que confirma que o código funciona com outros dados no mesmo formato.

```
In [ ]: # Outro tipo de alteração: o professor de uma aula de H0 fica indisponível
# exatamente nesse tempo.
_turma, _disc, _dia, _tempo = sorted(H0)[0]
_prof = next(d["professor"] for d in disciplinas if d["disciplina"] ==
_disc)
excecoes_extra = disponibilidade_excecoes + [
    {"professor": _prof, "dia": _dia, "periodo": _tempo}]

_e1, H_zero_extra, tempo_zero_extra = gerar_do_zero(
    turmas, disciplinas, salas, excecoes_extra)
_e2, H_inc_extra, tempo_inc_extra = gerar_incremental(
    H0, turmas, disciplinas, salas, excecoes_extra)

mo.md(f"""
Cenário: {_prof} fica indisponível em {_dia}, tempo {_tempo}.

| | tempo (s) | aulas que mudam vs H0 |
|---|---|---|
| do zero | {tempo_zero_extra:.3f} | {mudancas(H0, H_zero_extra)} |
| incremental | {tempo_inc_extra:.3f} | {mudancas(H0, H_inc_extra)} |
""")
```

```
In [ ]: dados_teste = ler_dados("dados_teste")
_estado, H_teste, _tempo = gerar_do_zero(*dados_teste)
(_estado, len(H_teste), verificar(H_teste, *dados_teste))
```

7. Conclusões e limitações

Nos cenários que testámos, o incremental mudou muito menos aulas do que refazer o horário do zero. O H0, o H1 e o horário do conjunto de teste passaram no verificador.

O trabalho tem limites. O modelo conta as salas por tipo e não atribui salas concretas, por isso só medimos mudanças de tempo e não de sala. O H1 incremental não otimiza os buracos. A regra R7 assume uma única linha de sala normal no ficheiro de salas. Com duas turmas a diferença de tempo entre as duas maneiras é pequena e só temos uma medição, por isso não tirámos conclusões sobre a rapidez. Ficaram por fazer os extras opcionais, as preferências dos professores e o teste com mais turmas.

8. Escolhas técnicas

O enunciado deixa três escolhas ao nosso critério. Estas são as nossas e o porquê.

Modelação: CP-SAT, do OR-Tools. É o que a disciplina sugere e serve bem a este problema. Aceita variáveis de sim ou não, somas com limites, um objetivo para minimizar e um horário de partida como pista, que é o que o R9 usa.

Leitura dos dados: módulo `csv`. Usámos o `csv` da biblioteca standard e não o `pandas` porque os ficheiros são pequenos e simples, e assim não precisamos de mais uma biblioteca.

Apresentação: tabelas em markdown. Cada horário aparece numa grelha por turma, com os dias nas colunas e os tempos nas linhas. No R9, o antes e o depois ficam lado a lado, com as aulas que mudam a negrito.